

MATAR UN PROCESO

A la hora de ejecutar el proyecto, en caso de que esté el block de notas abierto, lo cerrará. Si no está abierto, mostrará por consola un mensaje avisando de que no se ha encontrado.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.List;

public class KillProcessByName {

    public static void main(String[] args) {
        String processName = "Notepad.exe"; // Cambia esto al nombre del
        proceso que deseas terminar
        try {
            // Obtenemos el listado de procesos
            List<String> pids = getProcessIds(processName);

            if (pids.isEmpty()) {
                System.out.println("No se encontraron procesos con el nombre:
" + processName);
            } else {
                // Finalizamos los procesos con los PIDs obtenidos
                for (String pid : pids) {
                    terminateProcess(pid);
                    System.out.println("Proceso con PID " + pid + "
terminado.");
                }
            }
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    // Método para obtener los PIDs de los procesos con el nombre dado
    private static List<String> getProcessIds(String processName) throws
Exception {
        List<String> pids = new ArrayList<>();

        // Ejecutamos el comando tasklist para listar procesos (Windows)
        ProcessBuilder builder = new ProcessBuilder("tasklist.exe", "/fo",
"csv", "/nh");
        Process process = builder.start();
```

```

        // Leemos la salida del comando
        BufferedReader reader = new BufferedReader(new
InputStreamReader(process.getInputStream()));
        String line;
        while ((line = reader.readLine()) != null) {
            // Filtramos las líneas que contienen el nombre del proceso
            if (line.contains(processName)) {
                // El PID es el segundo campo en la salida de tasklist (CSV)
                String[] fields = line.split("\\,\\");
                if (fields.length > 1) {
                    String pid = fields[1].replaceAll("\\\\", "");
                    pids.add(pid);
                }
            }
        }
        return pids;
    }

    // Método para terminar un proceso basado en su PID
    private static void terminateProcess(String pid) throws Exception {
        // Ejecutamos el comando taskkill para finalizar el proceso (Windows)
        ProcessBuilder builder = new ProcessBuilder("taskkill", "/PID", pid,
"/F");
        builder.start();
    }
}

```

SUBPROCESO

Ejemplo que espera a que el proceso hijo finalice y muestra por consola el resultado.

```
import java.io.BufferedReader;
import java.io.InputStreamReader;

public class SubprocessExample {

    public static void main(String[] args) {
        try {
            // Crear un proceso que ejecute el comando "ping" para comprobar
            la conexión a una dirección
            ProcessBuilder builder = new ProcessBuilder("ping", "google.com");

            // Iniciar el proceso
            Process process = builder.start();

            // Capturar la salida del proceso
            BufferedReader reader = new BufferedReader(new
            InputStreamReader(process.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }

            // Esperar a que el proceso termine y obtener el código de salida
            int exitCode = process.waitFor();
            System.out.println("Proceso terminado con código de salida: " +
            exitCode);

        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    }ObjectInputStream entrada = new ObjectInputStream(cliente.getInputStream());
    entrada.readObject();
    ObjectOutputStream salida = new ObjectOutputStream(cliente.getOutputStream());
    salida.writeObject();
    entrada.close();
    salida.close();
    cliente.close();
```